

Atividade Domiciliar

Testes e Qualidade em uma Esteira DevOps CI/CD com Docker

Disciplina: Testes e Qualidade de Software

Item	Descrição
Modalidade	Atividade individual domiciliar com apresentação em sala
Produto esperado	Repositório Git com aplicação, Dockerfile, pipeline CI/CD, testes automatizados e evidências
Apresentação	10 minutos

1. Objetivo da atividade

O aluno deverá criar ou adaptar uma pequena aplicação de software, empacotá-la em container Docker e configurar uma esteira DevOps CI/CD que execute verificações automáticas de qualidade, testes automatizados e geração de evidências. Ao final, deverá apresentar em sala o funcionamento da esteira e explicar como ela contribui para a qualidade do produto e do processo de software.

2. Descrição geral

A atividade deve utilizar Docker e uma ferramenta de integração contínua, como GitHub Actions, GitLab CI/CD, Jenkins, Azure DevOps ou equivalente. A aplicação pode ser simples; o foco principal é demonstrar o uso de qualidade, testes, automação, containers e CI/CD no desenvolvimento moderno de software.

Sugestões de aplicação:

- API de cadastro de alunos;
- Sistema de tarefas;
- Calculadora de média escolar;
- CRUD de produtos, livros ou disciplinas;
- Sistema de login fictício;
- Loja virtual simplificada.

3. Requisitos obrigatórios

3.1 Repositório Git

O projeto deverá estar em um repositório Git contendo:

- Código-fonte da aplicação;
- Arquivo Dockerfile;
- Arquivo docker-compose.yml, se aplicável;
- Arquivo de configuração da pipeline CI/CD;
- Testes automatizados;
- README explicando o projeto;
- Evidências da execução da esteira.

3.2 Uso obrigatório de Docker

O projeto deverá ser executado em container Docker. O aluno deverá criar um Dockerfile para empacotar a aplicação.

Exemplo de estrutura:

```
meu-projeto/
├── src/
├── tests/
├── Dockerfile
├── docker-compose.yml
├── README.md
└── .github/workflows/ci.yml
```

A aplicação deverá ser executável com comandos semelhantes a:

```
docker build -t minha-aplicacao .
docker run minha-aplicacao
```

Ou, usando Docker Compose:

```
docker compose up --build
```

3.3 Docker Compose

Quando a aplicação utilizar banco de dados, API externa simulada ou múltiplos serviços, o aluno deverá usar docker-compose.yml.

Exemplos de serviços possíveis:

- aplicação;
- banco de dados;
- serviço de testes;
- ferramenta de análise;
- ambiente de execução.

Exemplo conceitual:

```
services:
  app:
    build: .
    ports:
      - "3000:3000"

  tests:
    build: .
    command: npm test
    depends_on:
      - app
```

4. Esteira CI/CD obrigatória

A pipeline deverá executar, no mínimo, as etapas a seguir.

Etapa 1 — Checkout do código

A esteira deverá baixar o código-fonte do repositório.

Exemplo:

```
- uses: actions/checkout@v4
```

Etapa 2 — Build da imagem Docker

A pipeline deverá construir a imagem Docker da aplicação, verificando se a aplicação consegue ser empacotada corretamente.

Exemplo:

```
docker build -t app-teste-qualidade .
```

Etapa 3 — Execução de análise estática ou linter

A esteira deverá executar uma verificação de qualidade estática. Essa etapa pode ser executada dentro do container.

Exemplos de ferramentas:

- ESLint;
- Pylint;
- Flake8;
- Checkstyle;
- SonarQube ou SonarCloud;
- SpotBugs.

Exemplos de execução:

```
docker run app-teste-qualidade npm run lint
docker run app-teste-qualidade pytest --flake8
```

Essa parte está relacionada a testes estáticos, prevenção de defeitos e qualidade do processo.

Etapa 4 — Testes automatizados dentro do container

A pipeline deverá executar testes automatizados dentro do ambiente Docker.

O aluno deverá implementar pelo menos:

- 3 testes de unidade;
- 1 teste de integração ou API; (opcional, mas desejável)
- 1 teste de regressão automatizado. (opcional, mas desejável)

Exemplos de execução:

```
docker run app-teste-qualidade npm test
docker run app-teste-qualidade pytest
```

A ideia é garantir que os testes sejam executados em um ambiente padronizado, evitando o problema de “funciona na minha máquina”.

Etapa 5 — Geração de evidências

A esteira deverá produzir evidências da qualidade do software, como:

- log da pipeline;
- relatório de testes;
- relatório de cobertura;
- screenshot da execução;
- badge (ícone) de status da pipeline;
- relatório do linter;
- evidência de falha e correção.

Exemplos:

```
pytest --junitxml=report.xml --cov=src
```

```
npm test -- --coverage
```

Etapa 6 — Simulação de falha e correção

O aluno deverá demonstrar que a esteira consegue detectar defeitos. Para isso, deverá:

1. Criar uma alteração que quebre um teste;
2. Executar a pipeline;
3. Registrar a falha;
4. Corrigir o problema;
5. Executar a pipeline novamente;
6. Apresentar a execução corrigida.

Essa etapa deve estar documentada no README ou em relatório curto.

5. Entregáveis

O aluno deverá entregar:

7. Link do repositório Git;
8. Código-fonte da aplicação;
9. Dockerfile funcional;
10. docker-compose.yml, se utilizado;
11. Arquivo da pipeline CI/CD;
12. Testes automatizados;
13. Relatório ou README.

O relatório ou README deverá conter:

- objetivo da aplicação;
- tecnologias utilizadas;
- como executar com Docker;
- como executar os testes;
- etapas da pipeline;
- tipos de testes aplicados;
- evidências de execução;
- falha simulada e correção;
- conclusão sobre qualidade e DevOps.

6. Apresentação em sala

O aluno deverá apresentar em aproximadamente 8 a 12 minutos.

Parte da apresentação	O que demonstrar
1. Apresentação da aplicação	Nome do projeto, problema resolvido, funcionalidades principais e tecnologias utilizadas.
2. Uso de Docker	Demonstrar Dockerfile, build da imagem, execução do container e Docker Compose, se houver.
3. Testes implementados	Explicar testes unitários, integração/API, regressão e análise estática/linter.
4. Esteira CI/CD	Mostrar a pipeline e explicar checkout, build, análise estática, testes, relatórios e resultado final.
5. Falha simulada	Apresentar o erro criado, o teste que falhou, a detecção pela pipeline e a correção.
6. Conclusão	Explicar como Docker, CI/CD e automação de testes contribuem para a qualidade.

7. Modelo de README esperado

```
# Nome do Projeto

## Descrição
Explique brevemente o sistema desenvolvido.

## Tecnologias utilizadas
- Linguagem:
- Framework:
- Ferramenta de testes:
- Docker:
- CI/CD:

## Funcionalidades
- Funcionalidade 1
- Funcionalidade 2
- Funcionalidade 3

## Como executar com Docker

### Build da imagem
docker build -t nome-da-aplicacao .

### Execução da aplicação
docker run -p 3000:3000 nome-da-aplicacao

### Execução com Docker Compose
docker compose up --build

## Como executar os testes
docker run nome-da-aplicacao npm test
# ou
docker run nome-da-aplicacao pytest

## Pipeline CI/CD
A esteira executa as seguintes etapas:
1. Checkout do código
2. Build da imagem Docker
3. Análise estática
4. Execução dos testes automatizados
5. Geração de relatório
6. Resultado final

## Tipos de testes implementados
- Testes unitários
```

- Teste de integração/API
- Teste de regressão
- Teste estático/linter

Evidências

Inserir prints ou links da execução da pipeline.

Falha simulada e correção

Explique qual erro foi criado, qual teste falhou e como o problema foi corrigido.

Conclusão

Explique como Docker, CI/CD e testes automatizados contribuem para a qualidade do software.

8. Exemplo de projeto simples

Tema: API de média escolar.

A aplicação deverá permitir:

- cadastrar aluno;
- registrar duas notas;
- calcular média;
- informar situação: aprovado ou reprovado;
- validar notas entre 0 e 10.

Testes possíveis:

- testar cálculo correto da média;
- testar aprovação;
- testar reprovação;
- testar nota inválida;
- testar regressão da regra de aprovação;
- testar endpoint de cálculo via API.

Esteira sugerida:

14. Baixar código;
15. Construir imagem Docker;
16. Executar linter;
17. Executar testes unitários;
18. Executar teste de integração/API;
19. Gerar relatório de cobertura;
20. Publicar evidências da pipeline.

9. Observações importantes

- Não publicar senhas, tokens, chaves de API ou dados reais no repositório.
- A aplicação pode ser simples, mas a esteira deve demonstrar claramente Docker, CI/CD, testes automatizados e evidências de qualidade.
- O aluno deverá conseguir explicar por que containers ajudam a padronizar o ambiente e reduzir falhas no processo de desenvolvimento.